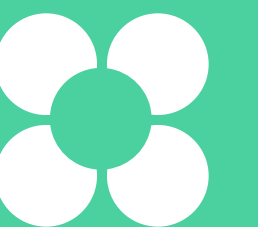


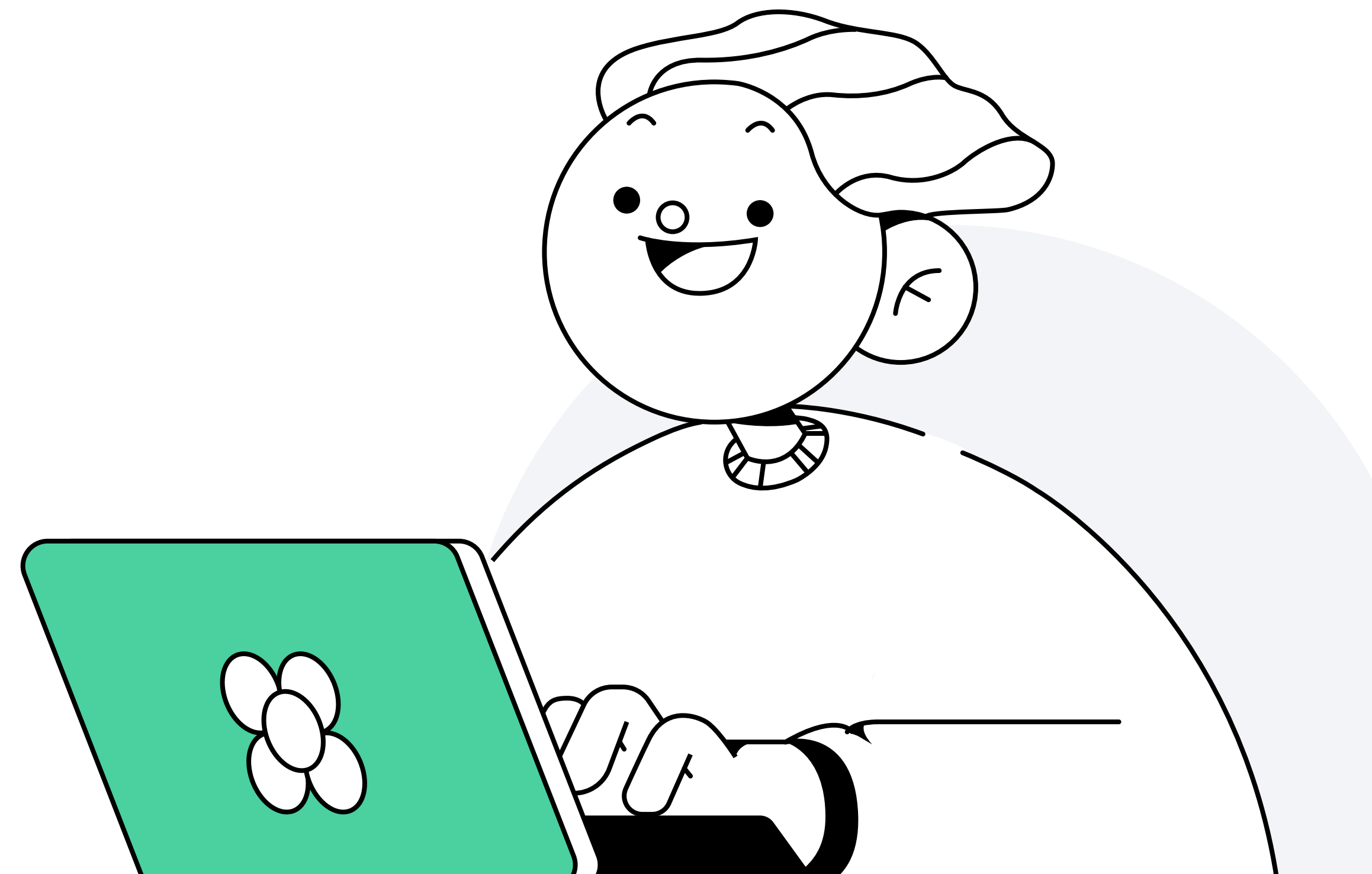
Системы мониторинга

Сергей Бывшев
Ведущий инженер автоматизации в «Метре квадратном»



План занятия

- 1 Понятие и назначение мониторинга
- 2 Типы мониторинга
- 3 Подходы к настройке мониторинга
- 4 Основные понятия
- 5 Zabbix
- 6 Prometheus
- 7 TICK



Понятие и назначение мониторинга

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Что такое мониторинг

Мониторинг — это сбор, обработка, агрегирование и отображение в реальном времени количественных и качественных показателей системы

Мониторинг позволяет улучшать или оставлять на приемлемом уровне качество обслуживания пользователей

Что такое мониторинг

На практике мониторинг чаще всего включает в себя:

- оценку работоспособности ПО
- оценку работоспособности оборудования
- бизнес-мониторинг
- мониторинг безопасности ИС

Зачем нужен мониторинг

- **Анализ долгосрочных тенденций** — получение качественных характеристик и обеспечение дальнейшей работоспособности системы. Например, размер БД и его близость к критическим значениям
- **Сравнение версий ПО** — насколько изменения ПО повлияли на качество обслуживания
- **Оповещение** — превентивное выявление возможных отклонений в качестве обслуживания и увеличение скорости реакции на сбои
- **Телеметрия текущей работоспособности приложения** — получение текущей оценки характеристик информационной системы в режиме real-time

Типы мониторинга

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Типы мониторинга

Чаще всего мониторинг разделяют на 2 типа:

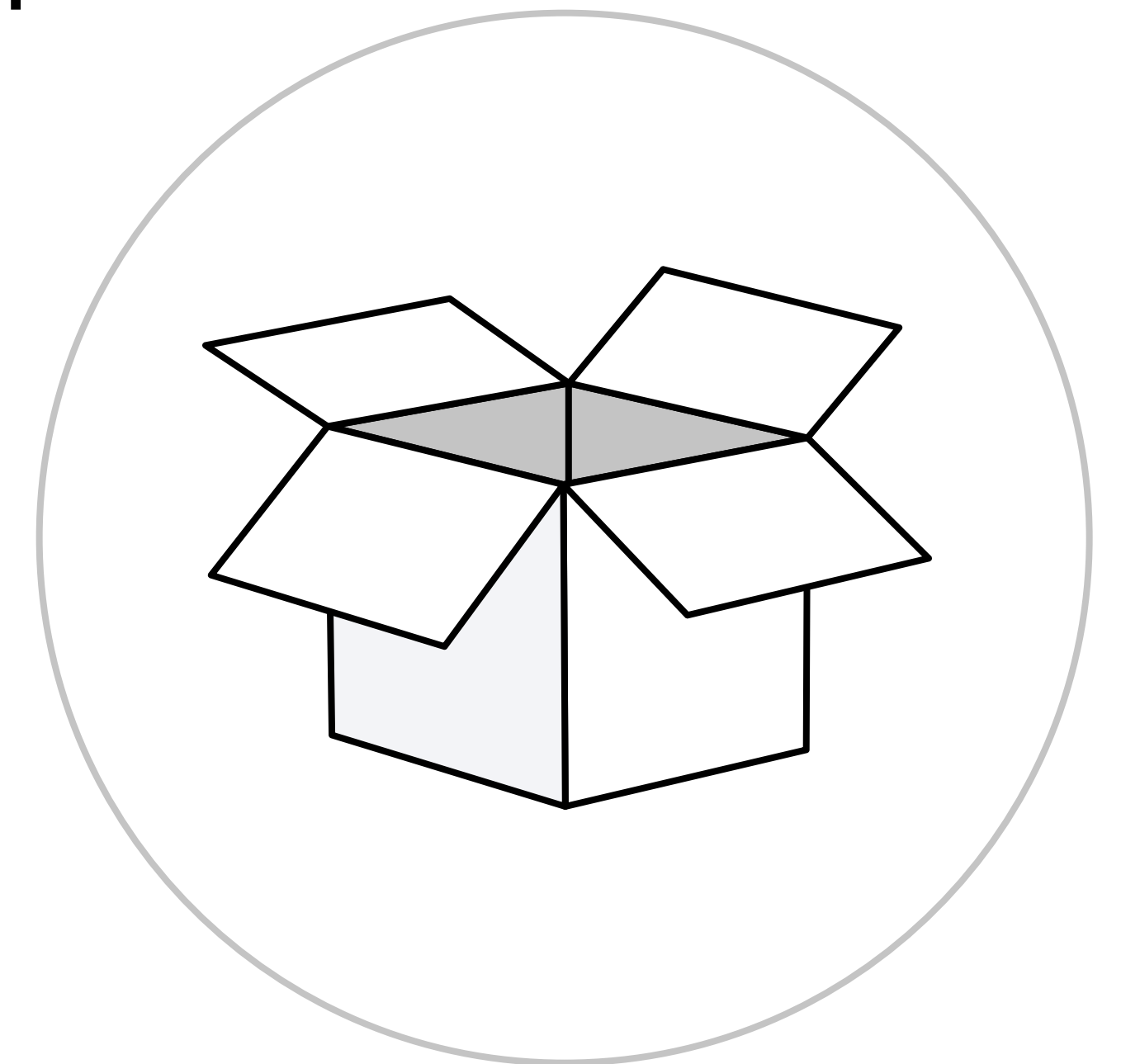
- **White-box monitoring** – наблюдение за системой изнутри. Сбор данных профилирования, логов, системные журналы
- **Black-box monitoring** – наблюдение извне: техническая поддержка или отдел тестирования собирают информацию о возникших ошибках системы



Типы мониторинга

Для DevOps-подхода важнее мониторинг white-box, так как только он позволяет спрогнозировать поведение системы и исправить проблемы до их возникновения.

Но про мониторинг black-box тоже не стоит забывать. Это симптоматическая диагностика, которая позволяет выявить редкие кейсы поломок системы



Деление мониторинга по доменам ответственности

Домены ответственности мониторинга можно разделить по системам сбора метрик:

- **система сбора временных рядов:** Prometheus, InfluxDB, Zabbix
- **система сбора логов:** Elastic Stack, Graylog, Grafana Loki
- **система «Перехватчик ошибок»:** Sentry
- **система сбора трейсов:** Jaeger, Zipkin, OpenTelemetry

Деление мониторинга по доменам ответственности

Системы сбора временных рядов позволяют получать телеметрию в режиме реального времени и хранить данные в таблицах в виде «метка времени — значение».

Такие системы нужны, чтобы непрерывно наблюдать за изменениями в работоспособности системы.

Пример метрики — доступное место на файловой системе

timestamp	CPU LA 15
2020-08-06 15:32:00	1,23
2020-08-06 15:32:30	1,30
2020-08-06 15:33:00	1,20
2020-08-06 15:32:30	2,00

Деление мониторинга по доменам ответственности

Основные метрики для мониторинга
в системах временных рядов:

- **CPU LA**
- **RAM/swap**
- **IOPS**
- **inodes**
- **FS**

Деление мониторинга по доменам ответственности

Дополнительные метрики для мониторинга
в системах временных рядов:

- **Process liveness**
- **IOwait**
- **NetTraffic**
- **Custom?**

Деление мониторинга по доменам ответственности

Системы сбора логов позволяют агрегировать логи приложений, системные журналы и т. п.

Такие системы чаще всего нужны **для детального разбора появившихся проблем**, нахождения их частоты появления, разбора поведения системы в каких-то условиях.

Пример — логи балансировщика запросов

Деление мониторинга по доменам ответственности

**Система «Перехватчик ошибок» позволяет информировать
о возникающих ошибках ПО в режиме real-time**

Деление мониторинга по доменам ответственности

Для системы можно указать дополнительные характеристики, которые будут в информировании:

- слой (stage/production)
- hostname
- номер реплики приложения
- внутреннее состояние приложения

Деление мониторинга по доменам ответственности

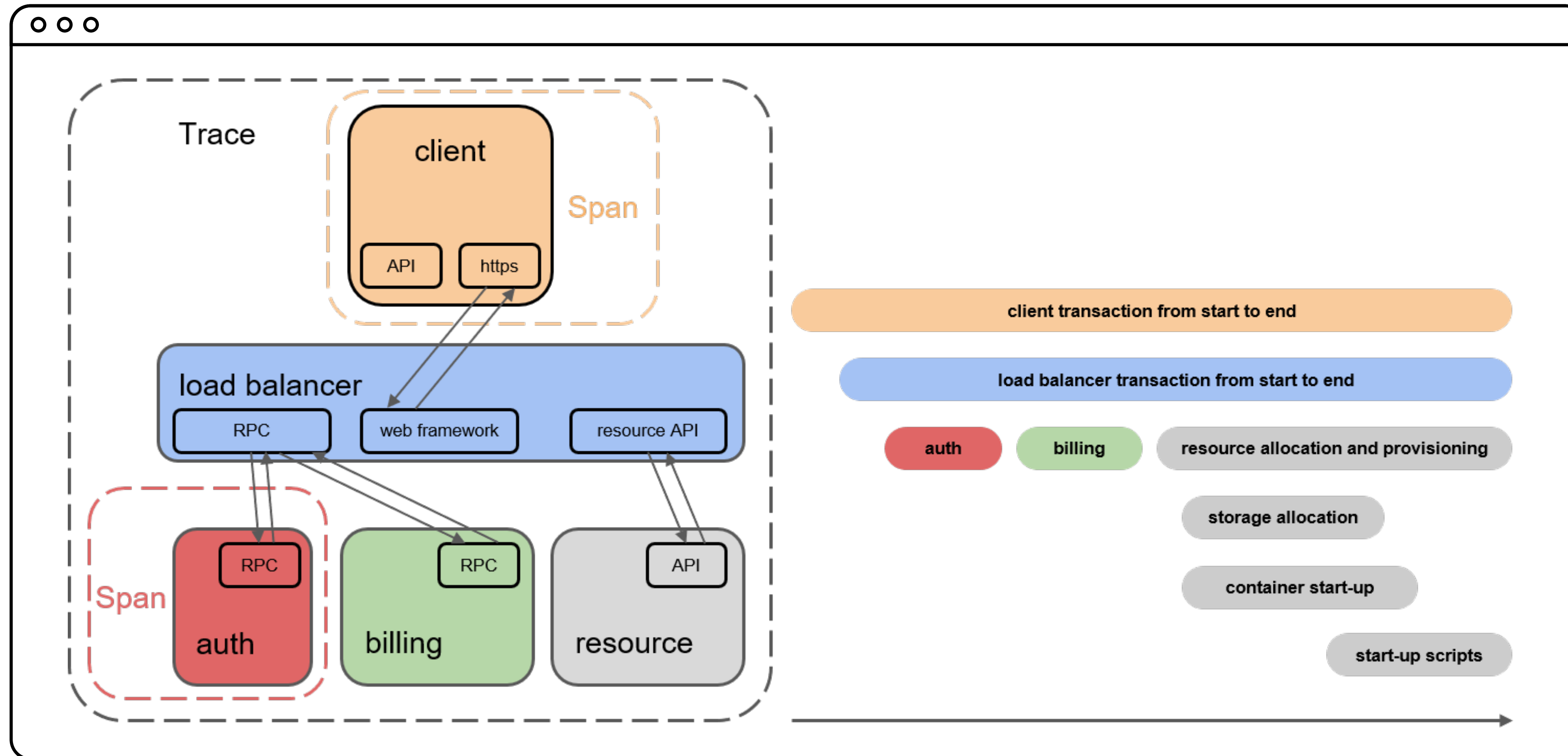
Основные метрики для мониторинга в системах
сбора логов и ловли ошибок:

- **Good events / All events**
- **Traffic**

Деление мониторинга по доменам ответственности

Системы сбора трейсов позволяют собирать и отображать весь путь прохождения клиентского запроса

Деление мониторинга по доменам ответственности



Подходы к настройке мониторинга

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Подходы при настройке мониторинга

Приведённые подходы не являются формальными и не будут тем самым «золотым молотком», который решит все проблемы.

Но желательно эти подходы знать и использовать при необходимости.

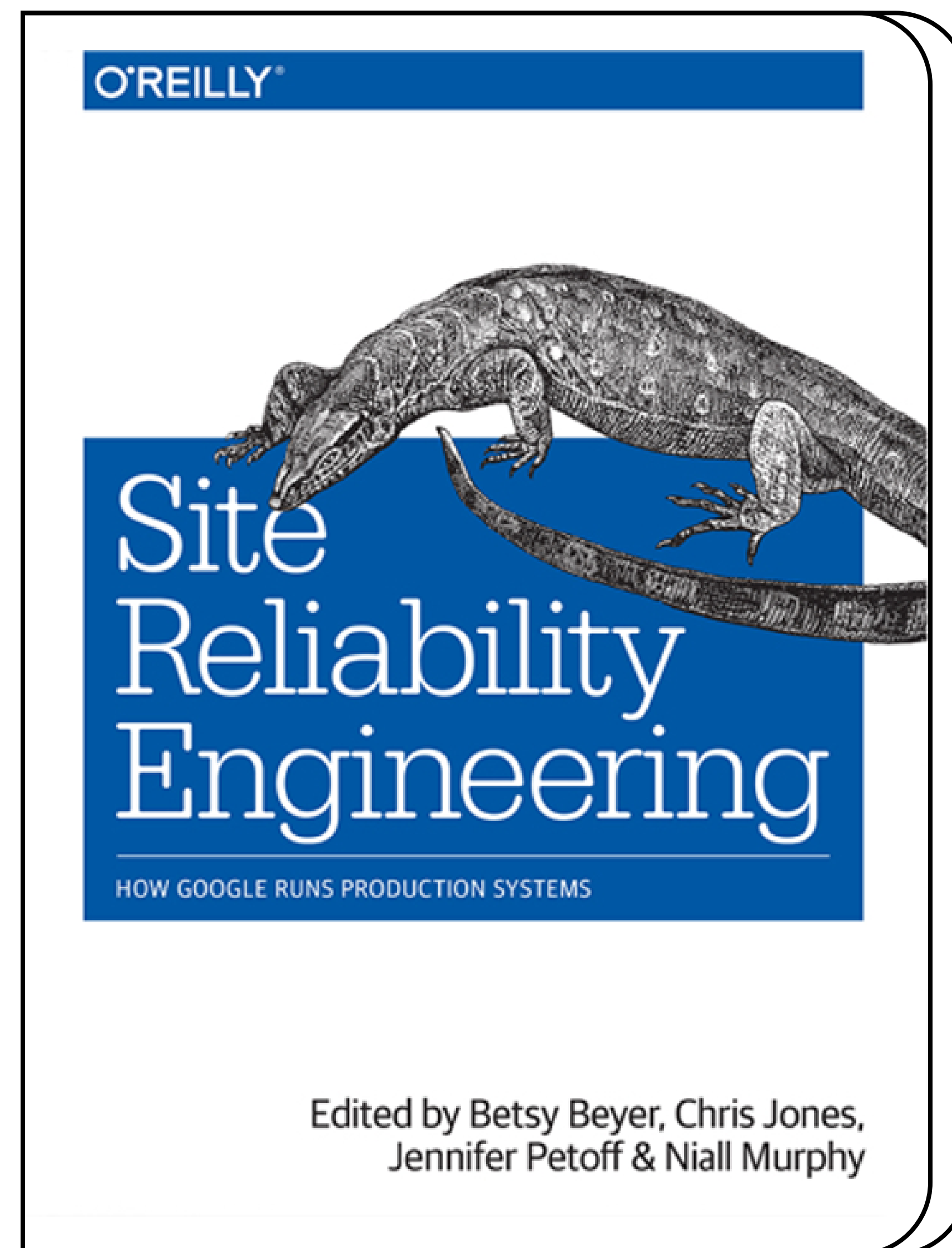
Они позволяют экономить на ресурсах, выделенных для мониторинга, связать мониторинг с бизнес-частью и улучшить качественную интерпретацию полученных метрик

Подходы при настройке мониторинга: SRE

Подход SRE разработал Google, он повсеместно внедряется в компаниях по всему миру.

SRE покрывает большинство задач **DevOps**, в том числе задачи мониторинга.

Один из его главных плюсов — он **сделан практикующими инженерами для практикующих инженеров** на основе многолетнего опыта



Подходы при настройке мониторинга: SRE

Основные тезисы подхода при настройке мониторинга:

- ставьте для мониторинга реальные задачи
- система мониторинга должна отвечать на два вопроса «Что сломалось?» и «Почему сломалось?»
- используйте четыре золотых сигнала
- не исследуйте только средние значения
- выберите подходящий уровень детализации

Подходы при настройке мониторинга: SRE

Четырьмя золотыми сигналами в подходе SRE называются:

- **Время отклика** — время, которое требуется для выполнения запроса
- **Величина трафика** — величина нагрузки, которая приходится на вашу систему. Для веб — это количество HTTP-запросов, а для потокового аудио — скорость передачи данных
- **Уровень ошибок** — количество или частота неуспешно выполненных запросов. Например, ответ 500 от HTTP-сервера
- **Степень загруженности** — показатель того, насколько полно загружен ваш сервис. Это мониторинг компонентов, которые покажут загруженность вашего сервиса. Для вычислений — это ЦПУ, для In-memory БД — RAM

Подходы при настройке мониторинга: USE

- **Utilisation** — насколько сильно загружен ресурс
- **Saturation** — длина очереди или время пребывания задач в очереди
- **Errors** — количество или частота неуспешно выполненных запросов. Например, ответ 500 от HTTP-сервера

Подходы при настройке мониторинга

- **SLO** — целевой уровень качества обслуживания.
Целевое значение или диапазон значений
- **SLA** — соглашение об уровне обслуживания. Явный или неявный контракт с внешними пользователями, включающий в себя последствия невыполнения SLO
- **SLI** — индикатор качества обслуживания. Конкретная величина предоставляемого обслуживания

Подходы при настройке мониторинга

Метрики SLO, SLA и SLI позволяют связать измеряемые **технические значения с бизнес-составляющей.**

Пример простого SLA

Если наш сайт отдаёт HTTP-коды 4xx/5xx, то мы должны произвести денежную компенсацию пользователю, если это не запланированное техническое обслуживание.

Пример SLO

В 99 % случаев мы должны отдавать HTTP-коды, отличные от 4xx/5xx.
1 % выделяется на техническое обслуживание.

Расчёт SLI может выглядеть таким образом:

$$SLI = (\text{summ_2xx_requests} + \text{summ_3xx_requests}) / (\text{summ_all_requests})$$

Основные понятия

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Системы мониторинга

В лекции мы рассмотрим наиболее популярные решения, которые применяются в подавляющем большинстве организаций:

- **Zabbix**
- **Prometheus**
- **TICK** (Telegraf, Influxdb, Chronograf, Kapacitor)

Существует множество систем мониторинга IT-систем, но они все в той или иной степени схожи с представленными

Подтипы систем мониторинга

Системы мониторинга можно разделить на 2 подтипа:
push-модель и **pull-модель**.

Эти подтипы характеризуют процесс сбора метрик
внутри системы мониторинга

Подтипы систем мониторинга

Push-модель подразумевает отправку данных с агентов (рабочих машин, с которых собираем мониторинг) в систему мониторинга, через вспомогательные службы или программы (обычно UDP)

Pull-модель подразумевает последовательный или параллельный сбор системой мониторинга с агентов накопленной информации из вспомогательных служб

Преимущества push-модели

Плюсы push-модели:

- ⊕ упрощение репликации данных в разные системы мониторинга или их резервные копии
- ⊕ более гибкая настройка отправки пакетов данных с метриками
- ⊕ UDP — это менее затратный способ передачи данных, из-за чего может возрасти производительность сбора метрик

Преимущества pull-модели

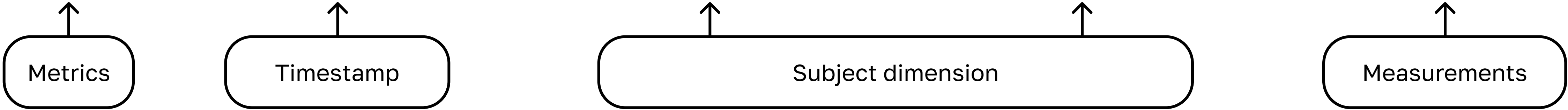
Плюсы pull-модели:

- ⊕ легче контролировать подлинность данных
- ⊕ можно настроить единый proxy server до всех агентов с TLS
- ⊕ упрощённая отладка получения данных с агентов

Time series database

TSDb — база данных, которая чаще всего используется в системах мониторинга. Эти системы характеризуются возможностью эффективного хранения пар «метка времени — значение»

metric	timestamp	cluster	hostname	metric value
cpu	2015-04-28T17:50:00Z	Cluster-A	host-a	10
cpu	2015-04-28T17:50:10Z	Cluster-A	host-b	20
cpu	2015-04-28T17:50:20Z	Cluster-A	host-a	5
iops	Гос. клиника	Cluster-A	host-a	10
iops	Гос. клиника	Cluster-A	host-b	30
iops	Статейник	Cluster-A	host-a	8



Zabbix

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Zabbix

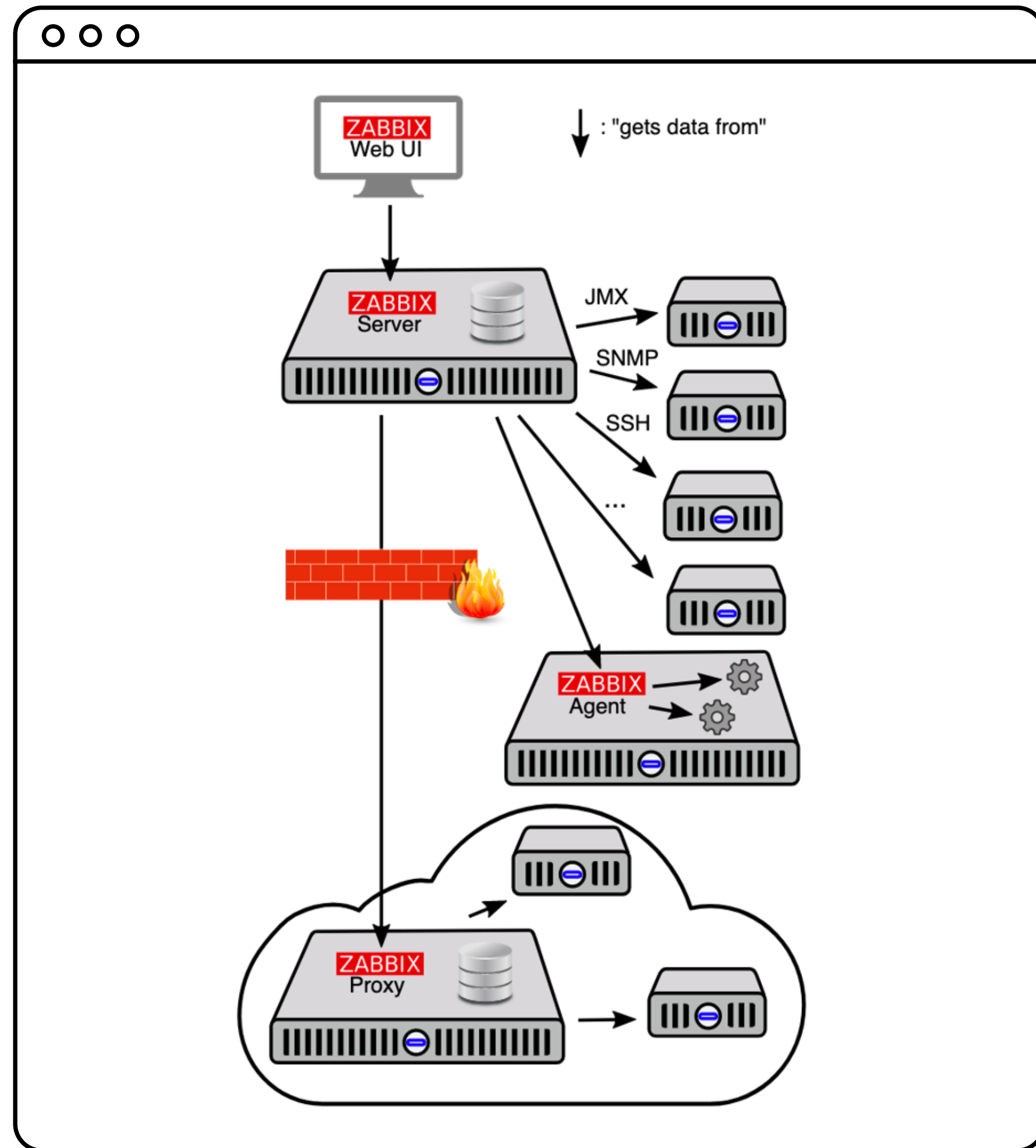
Zabbix – эффективная и зарекомендовавшая себя система мониторинга. Работает в соответствии с **push-** и **pull-моделью**.

Набор компонентов стека Zabbix:

- **Agent** – агент для сборки метрик с хост-машин и передачи в систему хранения
- **Server** – хранение данных конфигурации стека и статистики
- **Database** – хранение метрик мониторинга приложений
- **Proxy** – средство оптимизации нагрузки на server-компонент
- **Web UI** – веб-интерфейс для менеджмента данных, их визуализации, конфигурирования стека и настроек правил оповещения



Zabbix



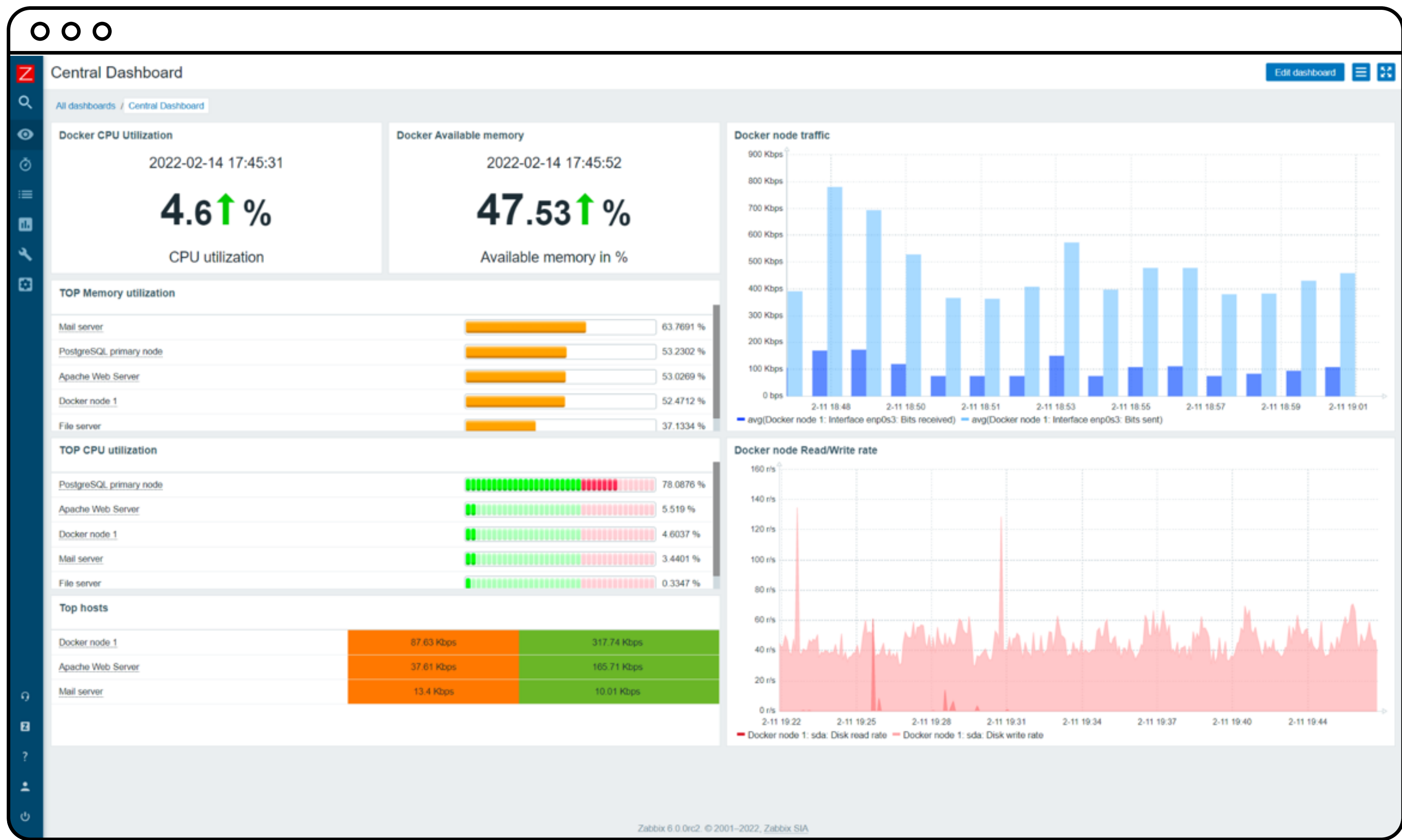
Zabbix

Механизм дуализма pull- и push-моделей в Zabbix заключается в настраиваемых активных и пассивных проверках.

- При **активных проверках** указываются данные для непрерывного наблюдения, которые агент перенаправляет серверу
- При **пассивных проверках** набор данных, которые собираются и хранятся на агенте, отправляется на сервер только по запросу

Zabbix

Дашборд с метриками сервера



Prometheus

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

Prometheus

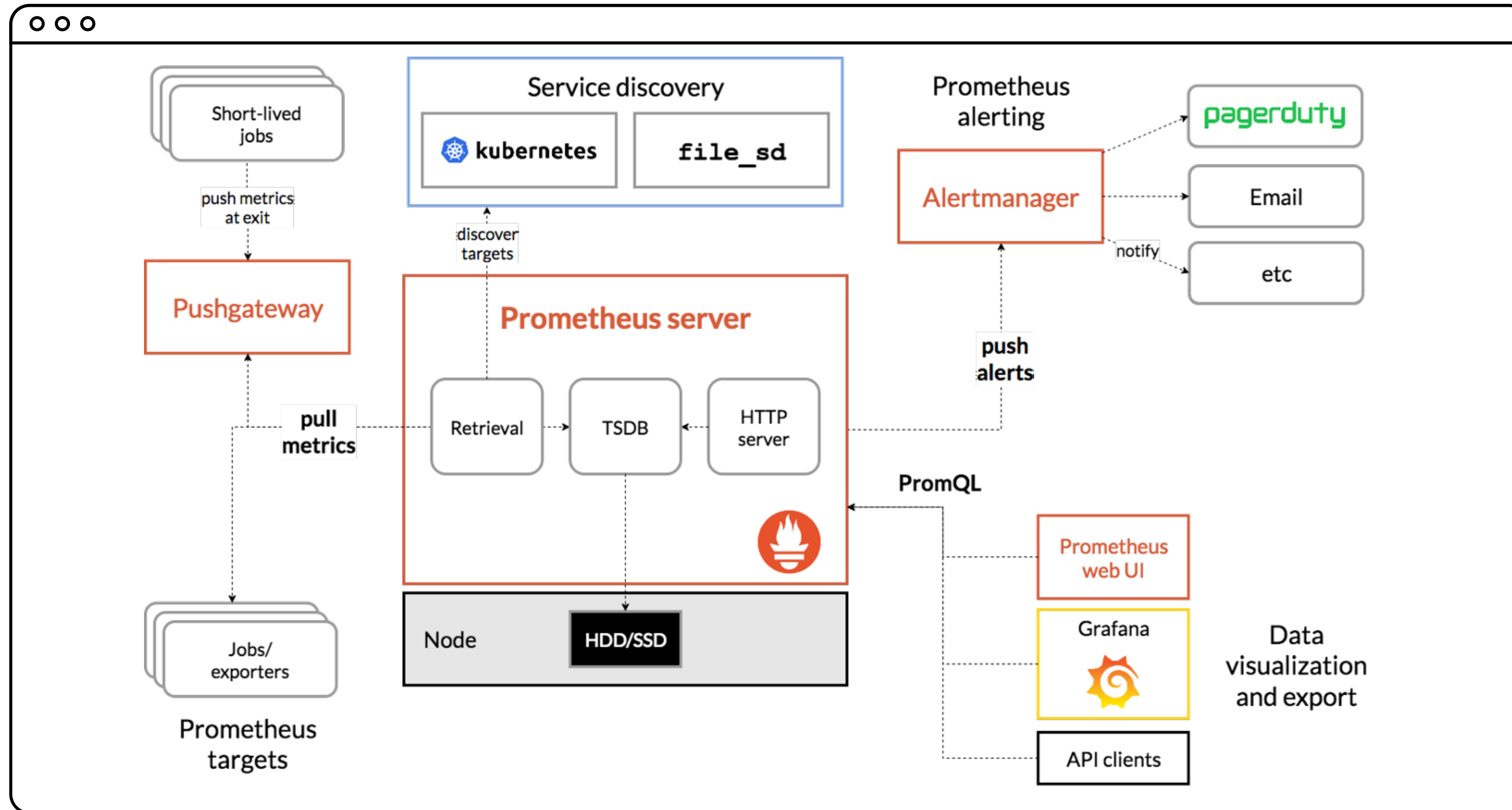
Prometheus — набор компонентов для эффективного мониторинга систем. Работает в соответствии с **pull-моделью**.

Набор компонентов стека Prometheus:

- Exporter — агент для сборки метрик с хост-машин и хранения их до сбора системой мониторинга
- Server — хранение данных и их менеджмент
- Web UI — веб-интерфейс для доступа к данным и конфигурирования системы
- Alert Manager — система оповещения



Prometheus



Prometheus

Почти весь набор компонентов Prometheus не требует дополнительных настроек и работает «из коробки».

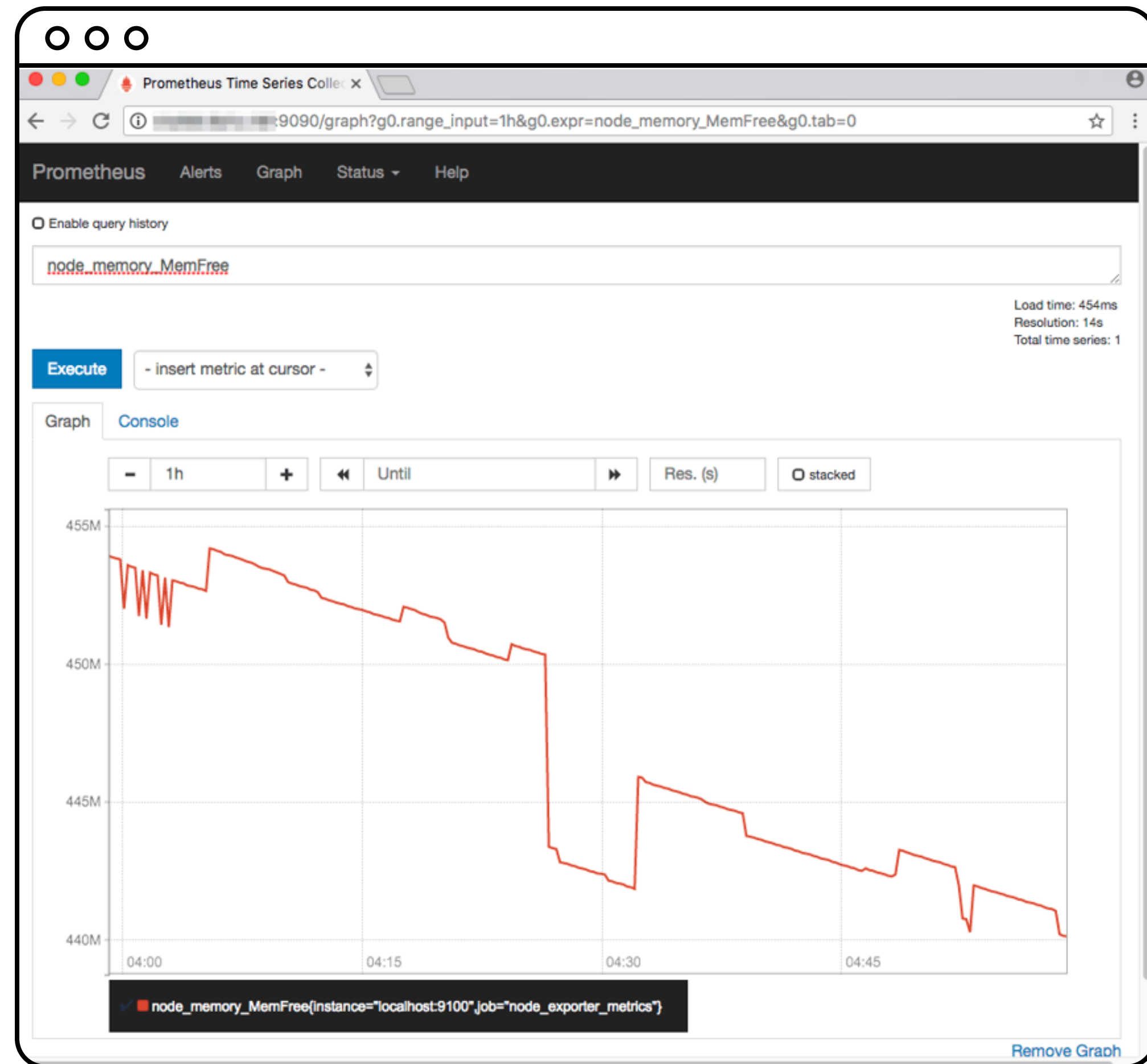
В соответствии с характеристиками работы pull-модели нам нужно лишь указать server-компоненту узлы для сбора метрик. Это можно сделать в конфигурационном файле (формат YAML):

```
global:
  scrape_interval: 15s
scrape_configs:
- job_name: node
  static_configs:
- targets: ['localhost:9100']
```



Prometheus

График изменения количества свободной памяти на сервере



TICK

Сергей Бывшев

Ведущий инженер автоматизации в «Метр квадратный»

TICK

TICK – набор компонентов для эффективного мониторинга систем.

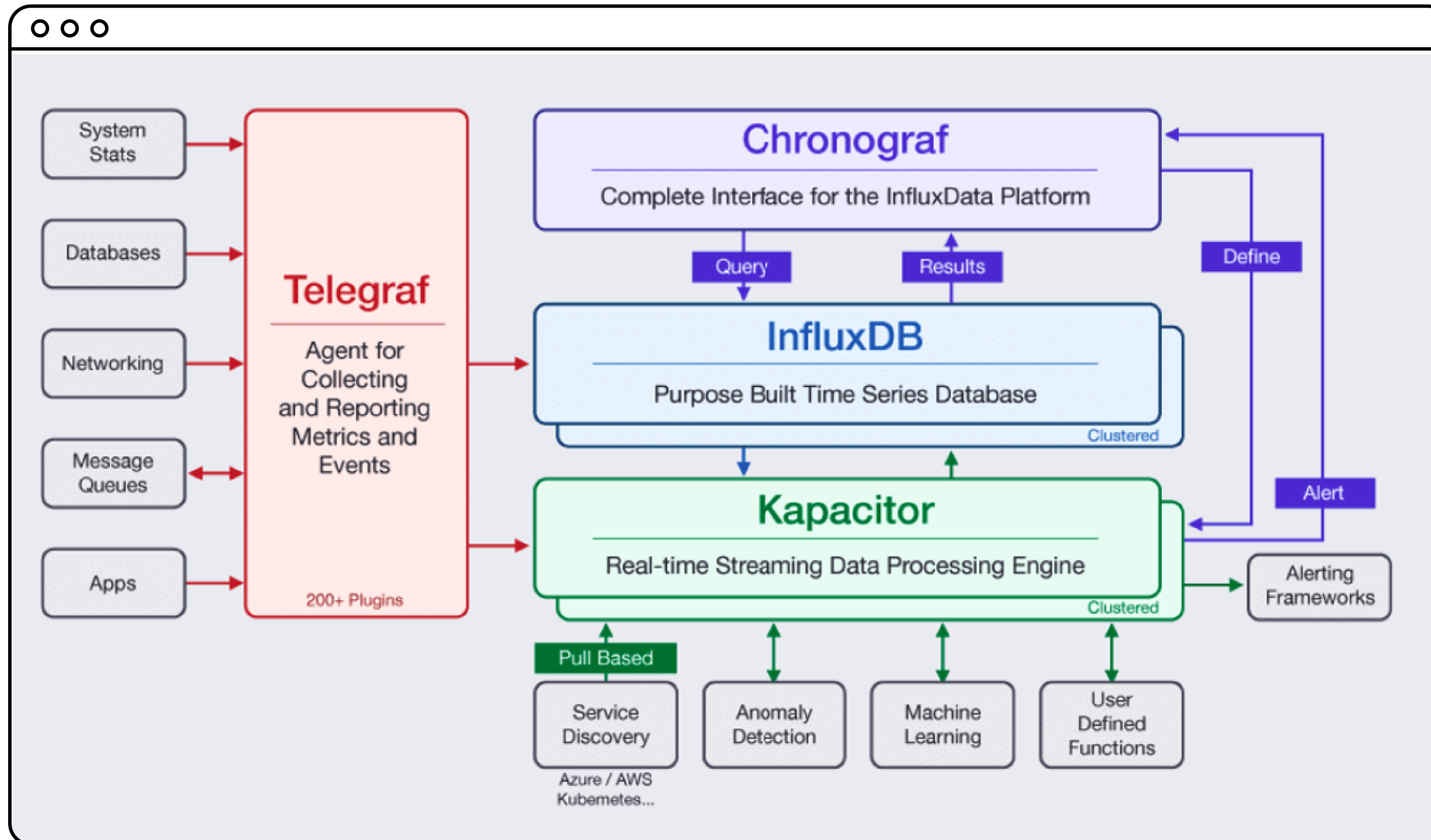
Работает в соответствии с **push-моделью**.

Набор компонентов стека TICK:

- Telegraf – агент для сборки метрик с хост-машин и отправки в TSDB
- InfluxDB – TSDB для хранения метрик
- Chronograf – компонент для визуализации и настройки данных TSDB
- Kapacitor – система для настройки правил оповещения

Этот стек технологий представляет собой «коробочную» версию системы мониторинга, которую можно использовать без дополнительных инструментов

TICK



TICK

Telegraf — Golang-приложение, которое собирает метрики и отправляет их в TSDB согласно конфигурации.

Конфигурирование осуществляется через файл **CONF**, где указываются входные параметры для сбора и конечные системы для отправки данных

TICK

Пример конфигурации для сборки «железных метрик» сервера и передачи их в **InfluxDB**:

```
[[inputs.cpu]]
    percpu = true
    totalcpu = true
    collect_cpu_time = false
    report_active = false
[[inputs.mem]]
[[inputs.disk]]
[[inputs.kernel]]
[[outputs.influxdb]]
    urls = ["udp://1.2.3.4:8089"]
```



TICK

InfluxDB – time series database, которая взаимодействует с данными через синтаксис, подобный SQL (InfluxQL).

Конфигурирование осуществляется через файл CONF, где указываются входные параметры хранения данных, производительности БД и сетевой интерфейс для доступа к данным



TICK

Chronograf — веб-интерфейс, который предоставляет доступ к данным InfluxDB и позволяет настраивать время жизни данных и правила оповещения Karasitor



TICK

Karacitor — компонент для настройки правил оповещения из системы мониторинга

TICK

Пример настроенного правила оповещения:

```
stream
  |from()
    .measurement('cpu_usage_idle')
    .groupBy('host')
  |window()
    .period(1m)
    .every(1m)
  |mean('value')
  |eval(lambda: 100.0 - "mean")
    .as('used')
  |alert()
    .message('{{ .Level }}: {{ .Name }}/{{ index .Tags
"host" }}')
    .warn(lambda: "used" > 70.0)
    .crit(lambda: "used" > 85.0)

// Slack
.slack()
.channel('#alerts')
```